

Assignment 4 — Solutions

Q1. Print all prime numbers from a list using loops

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True

numbers = [2, 3, 4, 5, 10, 13, 17, 20, 23, 25, 29, 1, 0]
primes = [n for n in numbers if is_prime(n)]
print("Prime numbers:", primes)

# Output: Prime numbers: [2, 3, 5, 13, 17, 23, 29]
```

Q2. Multiplication table (1–10) using nested loops

```
print(" ", end="")
for i in range(1, 11):
    print(f"{i:4}", end="")
print()
print("-" * 44)

for i in range(1, 11):
    print(f"{i:2} |", end="")
    for j in range(1, 11):
        print(f"{i*j:4}", end="")
    print()

# Output (partial):
# 1 2 3 4 5 6 7 8 9 10
# -----
# 1 | 1 2 3 4 5 6 7 8 9 10
# 2 | 2 4 6 8 10 12 14 16 18 20
# ...and so on up to 10
```

Q3. Function returning count of even numbers and sum of odd numbers

```
def analyze_list(lst):
    even_count = 0
```

```

odd_sum = 0

for num in lst:
    if num % 2 == 0:
        even_count += 1
    else:
        odd_sum += num
return even_count, odd_sum

nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
even_count, odd_sum = analyze_list(nums)
print(f"Count of even numbers: {even_count}") # 5
print(f"Sum of odd numbers: {odd_sum}") # 25

```

Q4. Simple Interest with default arguments

```

def simple_interest(principal, rate=5.0, time=2):
    si = (principal * rate * time) / 100
    return si

# Positional arguments
print(simple_interest(1000, 7, 3)) # 210.0

# Using default rate and time
print(simple_interest(1000)) # 100.0

# Keyword arguments
print(simple_interest(principal=5000, time=4)) # 1000.0
print(simple_interest(principal=2000, rate=8)) # 320.0

```

Q5 & Q6. Student class with grade method — create 2+ objects

```

class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def calculate_grade(self):
        if self.marks >= 90:
            return 'A+'
        elif self.marks >= 80:
            return 'A'
        elif self.marks >= 70:

```

```

return 'B'

elif self.marks >= 60:

return 'C'

elif self.marks >= 50:

return 'D'

else:

return 'F'


def display(self):

print(f"Student: {self.name} | Marks: {self.marks} | Grade: {self.calculate_grade()}")


# Create objects (Q6)
s1 = Student("Arjun Sharma", 92)
s2 = Student("Priya Mehta", 74)
s3 = Student("Rahul Verma", 55)


for s in [s1, s2, s3]:

s.display()


# Output:

# Student: Arjun Sharma | Marks: 92 | Grade: A+
# Student: Priya Mehta | Marks: 74 | Grade: B
# Student: Rahul Verma | Marks: 55 | Grade: D

```

Q7. BankAccount class with private balance (Encapsulation)

```

class BankAccount:

def __init__(self, owner, initial_balance=0):

self.__balance = initial_balance # private attribute

self.owner = owner


def deposit(self, amount):

if amount > 0:

self.__balance += amount

print(f"Deposited Rs.{amount}. New balance: Rs.{self.__balance}")

else:

print("Invalid deposit amount.")


def withdraw(self, amount):

if amount <= 0:

print("Invalid withdrawal amount.")

```

```

elif amount > self.__balance:
    print(f"Insufficient funds! Available: Rs.{self.__balance}")
else:
    self.__balance -= amount
    print(f"Withdrew Rs.{amount}. Remaining: Rs.{self.__balance}")

def get_balance(self):
    return self.__balance

acc = BankAccount("Arjun", 1000)
acc.deposit(500) # Deposited Rs.500. New balance: Rs.1500
acc.withdraw(200) # Withdrew Rs.200. Remaining: Rs.1300
acc.withdraw(2000) # Insufficient funds!
print(acc.get_balance()) # 1300
# print(acc.__balance) # AttributeError - cannot access directly

```

Q8. Division with ZeroDivisionError and ValueError handling

```

def safe_divide(a, b):
    try:
        a = float(a)
        b = float(b)
        result = a / b
        print(f"{a} / {b} = {result}")
    except ZeroDivisionError:
        print("Error: Division by zero is not allowed!")
    except ValueError:
        print("Error: Invalid input! Please enter numeric values.")

safe_divide(10, 2) # 10.0 / 2.0 = 5.0
safe_divide(10, 0) # Error: Division by zero is not allowed!
safe_divide("abc", 5) # Error: Invalid input! Please enter numeric values.

```

Q9. File handling — write student data and read it back

```

students_data = [
    ("Arjun Sharma", 92),
    ("Priya Mehta", 74),
    ("Rahul Verma", 55),
]

filename = "students.txt"

```

```
# Write student data to file
with open(filename, 'w') as f:
    f.write("Name,Marks\n")
    for name, marks in students_data:
        f.write(f"{name},{marks}\n")
    print(f"Data written to '{filename}'")
```

```
# Read and display the file content
print(f"\nContents of '{filename}':")
with open(filename, 'r') as f:
    for line in f:
        print(line.strip())
```

```
# Output:
# Data written to 'students.txt'
# Contents of 'students.txt':
# Name,Marks
# Arjun Sharma,92
# Priya Mehta,74
# Rahul Verma,55
```

Q10. Read numbers from file — calculate Total & Average

```
numbers_file = "numbers.txt"

# First create a sample file with numbers
with open(numbers_file, 'w') as f:
    f.write("10\n20\n30\n40\n50\n")

# Read numbers and calculate total & average
try:
    with open(numbers_file, 'r') as f:
        nums = [float(line.strip()) for line in f if line.strip()]
        total = sum(nums)
        average = total / len(nums)
        print(f"Numbers : {nums}")
        print(f"Total : {total}")
        print(f"Average : {average}")
except FileNotFoundError:
```

```
print("Error: File 'numbers.txt' does not exist!")
except ZeroDivisionError:
print("Error: File is empty, cannot calculate average.")

# Output:
# Numbers : [10.0, 20.0, 30.0, 40.0, 50.0]
# Total : 150.0
# Average : 30.0

# --- Demonstrating FileNotFoundError ---
try:
with open("missing_file.txt", 'r') as f:
data = f.read()
except FileNotFoundError:
print("Error: File 'missing_file.txt' does not exist!")
```